CS 193

Lecture 6: Github (Branching, Forking, Merging!!)

Announcements!


USB
COOKIE DROP
Enjoy a treat after the CS180/CS251 exams!
WEDS. 9/30 | LWSN entrance | 9:00 PM
BE THERE!
@purdueusb

HW 6 available!
Due **Sunday**, October 4th
@**11pm**.

Lecture 5 Review

Branching

When features are more than a commit

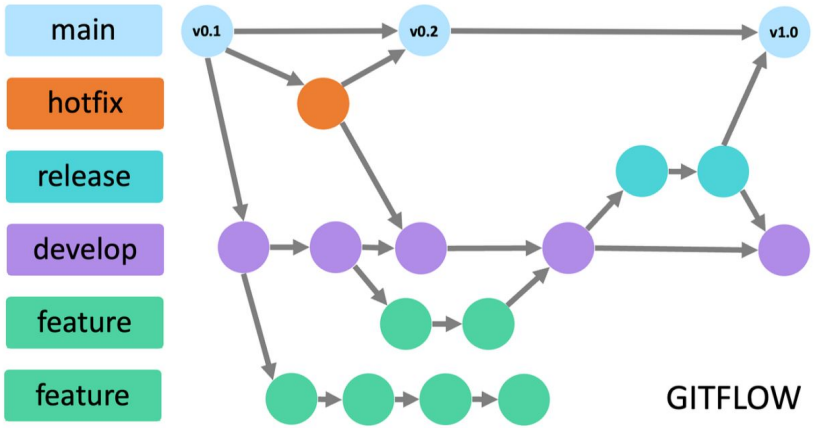
- Developing a new feature might require
  - multiple days
  - syncing with other team members
  - testing
  - debugging
- In the meantime, the rest of the team is working on other features and cannot wait for us
- A single commit will not do and trying to figure out if the new bugs are due to your code or code from someone else is untenable

Branching

Separating your changes to the codebase from the main line of code to work on new features, bug fixes, or experiments

Usually branches are made on:

- Features
- Bug fixes
- Experiments



https://media.licdn.com/dms/image/v2/D4D12AQH6ykM-AVoDKg/article-cover_image-shrink_720_1280/B4DZZW5EzAHAAM-/0/1745214525491?e=1792022400&v=beta&t=IEGYJXjZ8nTCZ8PxqLuNFuTctvfV4fECatgH3yDZHSQ

Why branch?

- Ideal for working in groups
    - ➜ Lets people work in parallel while minimizing merge conflicts
- Changes made to one branch do **not** affect other branches
    - ➜ Until you merge them



Common Branches

Main branch (previously called master!)

- The ‘main’ version of your project; if your project is published, this should be what the user sees
- Keep the main **clean** and **functioning**



Feature/bug fix branches

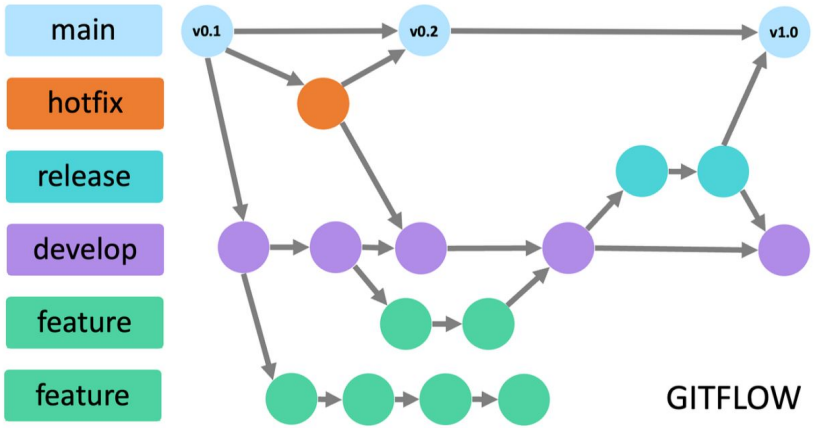
- **Named after feature being added or bug being fixed**
- Ideally, should only add one feature to the main branch
- Test out features in a feature branch, then merge to main once they’re functional!



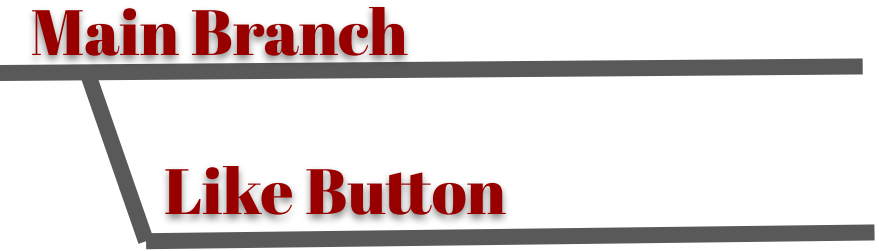
Common Branches

- In enterprise software
  - main is the latest version
  - develop is where current changes go into
  - Every feature has its own branch

- When development for a branch is completed
  - Gets reviewed
  - If it passes, gets merged to develop



Creating a Branch

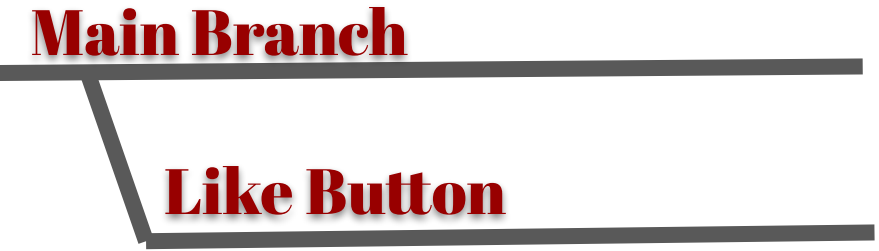



```
$ git branch branch_name
```

Create a new branch

*Doesn't switch to that branch, just creates it

Switching Branches



`$ git checkout branch_name`

Switches to *branch_name* (main branch)

*You can create a new branch and switch to it using `git checkout -b branch_name`

Code Reviews

Code Reviews

- Purpose: Review new code before it gets merged into the main branch

- This is industry standard!
  - Merging to main without getting approval might get you fired :(
  - It might also just be impossible

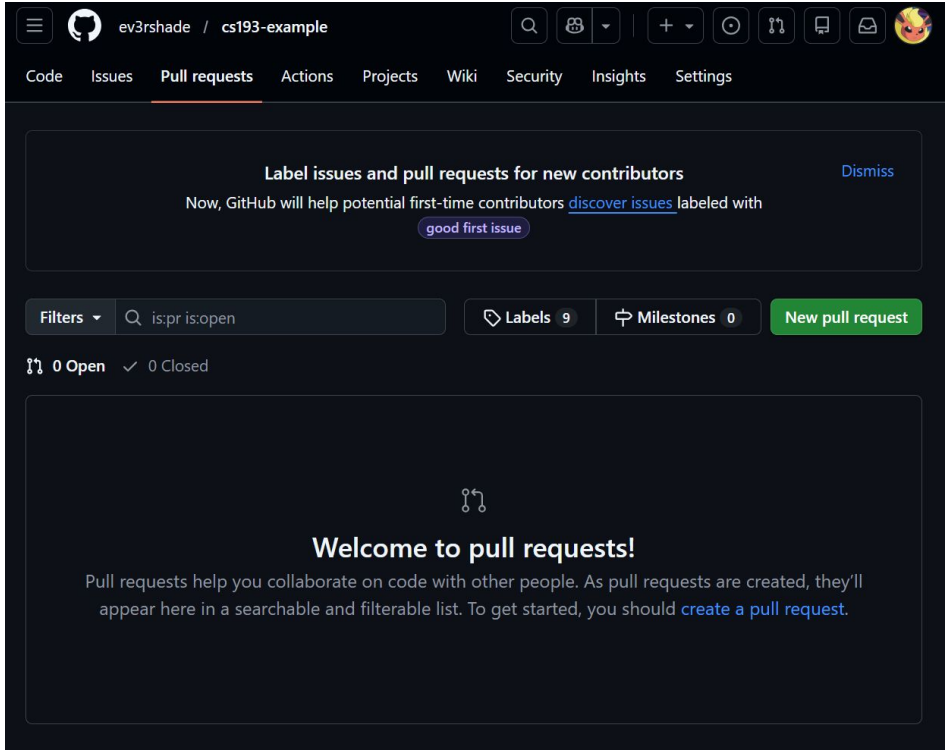


Code Reviews

- Process:
  - Reading through proposed changes (by yourself or others)
  - Scanning for noticable bugs
  - Suggestions/Comments

- Code is usually subjected to several rounds of this before being merged

- Similar feature in GitHub → **Pull Requests**



Git Tags

Tagging

- Every branch has a name
- And every commit has an ID
- But sometimes it is useful to give a friendly name to some important ones
  - For example version releases



Adding a tag

- You can add tags similarly to commits
  - `git tag v1.0.0 -m "Release 1.0.0"` Creates a new tag "v1.0.0" with a message
  - `git tag` Shows all tags
  - `git tag -d v1.0.0` Delete tag v1.0.0
  - `git push origin v1.0.0` Pushes the tag to origin

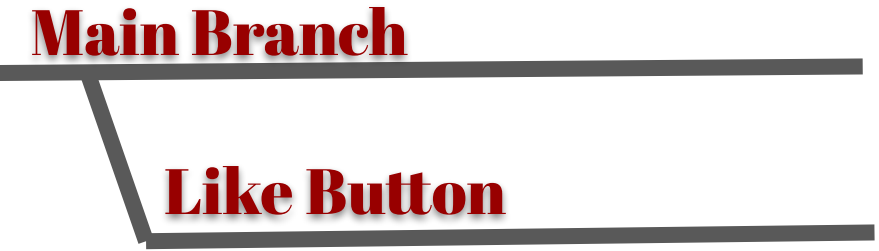
- Major versions: Breaking changes (python2 → python3)
- Minor version: New features
- Patch version: Small fixes for this minor version

Git commands: Branching Edition

Review of Git Commands

- **`$ git init`** → Creates an empty Git repository or reinitializes an existing one
- **`$ git add`** → Stages/prepares all the files you have changed in your local repo for committing
- **`$ git commit`** → Commits the staged changes you have made in your local repo
- **`$ git status`** → Displays the state of the working directory and the staging area
- **`$ git help <command>`** → Learn how to use a command

Creating a Branch

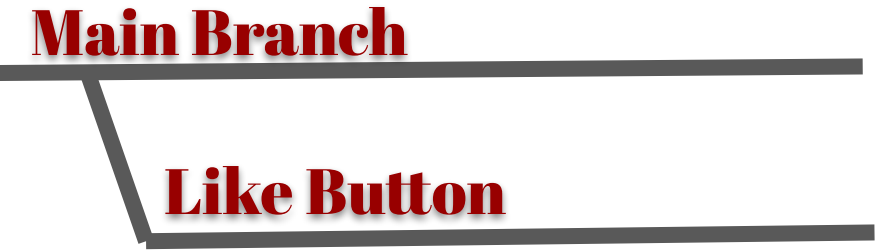



```
$ git branch branch_name
```

Create a new branch

*Doesn't switch to that branch, just creates it

Switching Branches

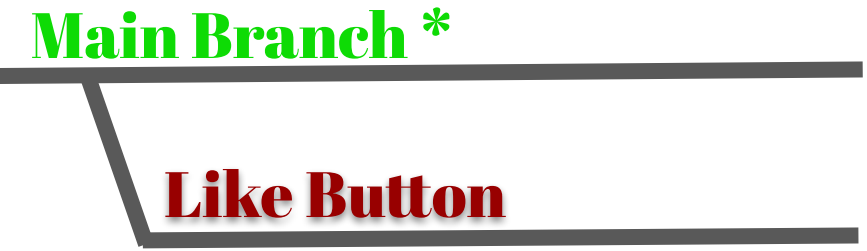


`$ git checkout branch_name`

Switches to *branch_name* (main branch)

*You can create a new branch and switch to it using `git checkout -b branch_name`

View Branches




```
$ git branch
```

Displays all branches, with current branch in green

Merging Branches




```
$ git merge branch_name
```

Merges *branch_name* into **the current branch you're on**

Rebasing

- Move an entire branch to begin on the tip of another branch
  - Incorporates all of the new commits in main into your branch
  - Re-writes the project history

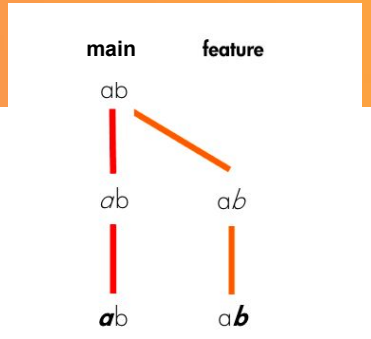
```
$ git checkout <branch name you want to rebase>

$ git rebase main
```

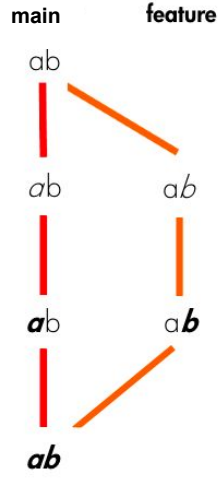
- Reckless rebasing
  - Issues arise where others have copies of the branch checked out that conflict with your altered copy

Rebasing vs. Merging

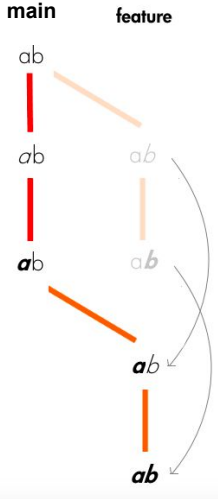
- Similarities
  - Both are used to integrate changes from one branch to another
- Differences
  - Merging results in combination of commits
  - Rebasing adds all changes in feature branch starting with last commit of the main branch



Merge



Rebase



The process of feature branching (remove slides)

Creating a feature branch

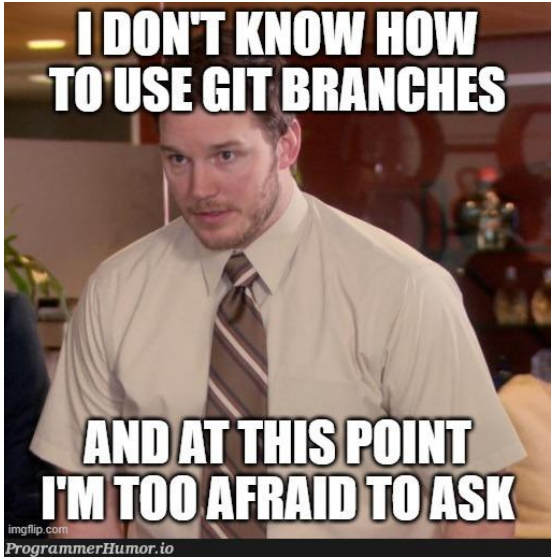
After cloning copy on terminal:

For a new branch…

- `git checkout -b <new_branch_name>`
  - Create and switch to branch <new_branch_name>
  - Changes made to a branch **do not** affect the code it was branched off of
- `git branch`
  - Check what branch you're in

To switch to an existing branch…

- `git checkout <branch_name>`
  - Your local changes will get overwritten by the code in the branch
    - (git will display and error message to help you store your changes → `git stash`)



Pushing your code on a branch

Instead of just running `git push`…

run `git push --set-upstream origin <branch_name>` since this is a brand new branch

- You only do this the first time (b/c it's a new branch)
- Afterwards, you can just run `git push`

Integrate changes to the main branch

- When you navigate back to github, you should see the following banner in your repository:



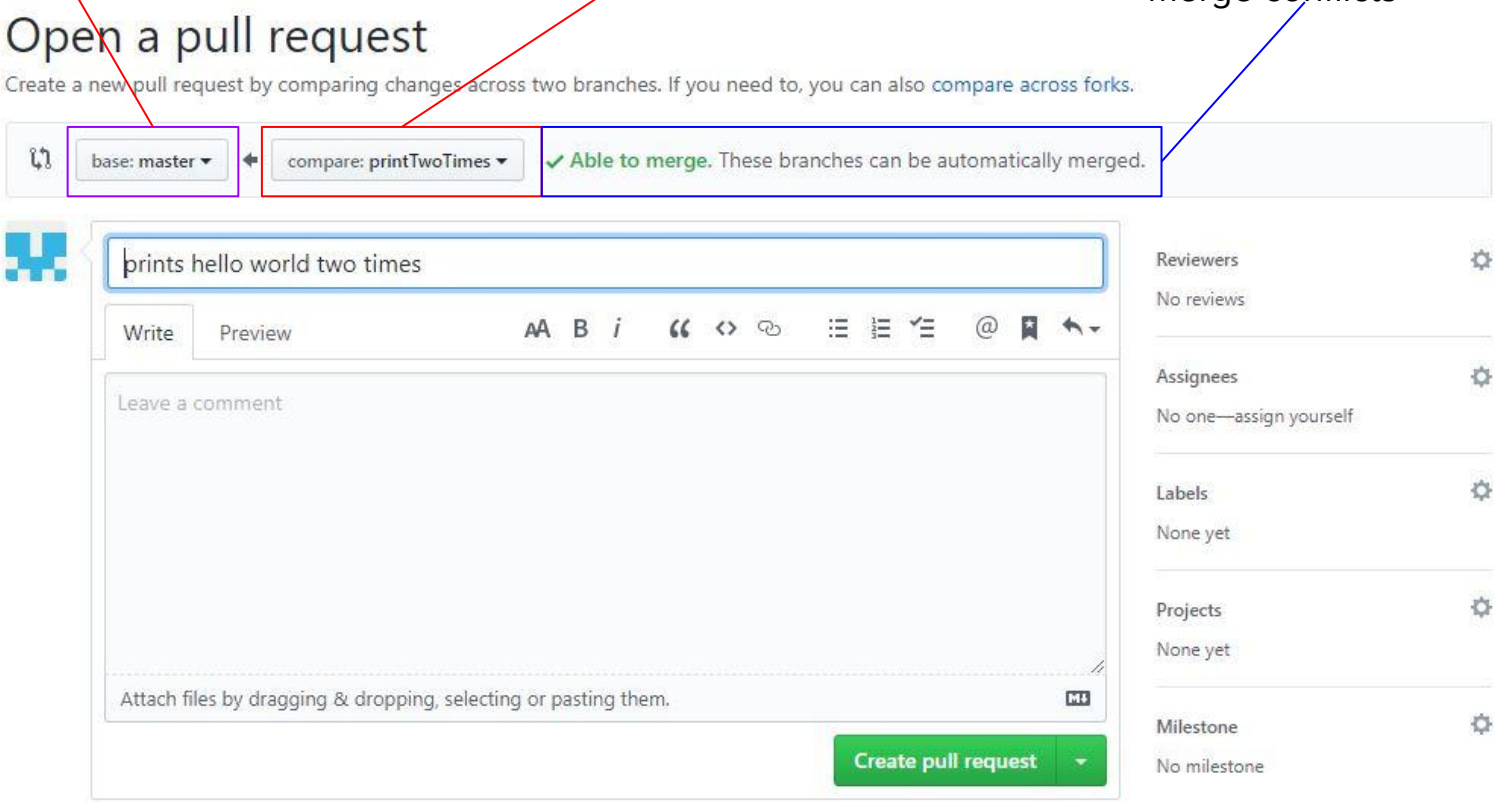
- Click ‘Compare & pull request’ to open a **pull request**

Opening a Pull Request

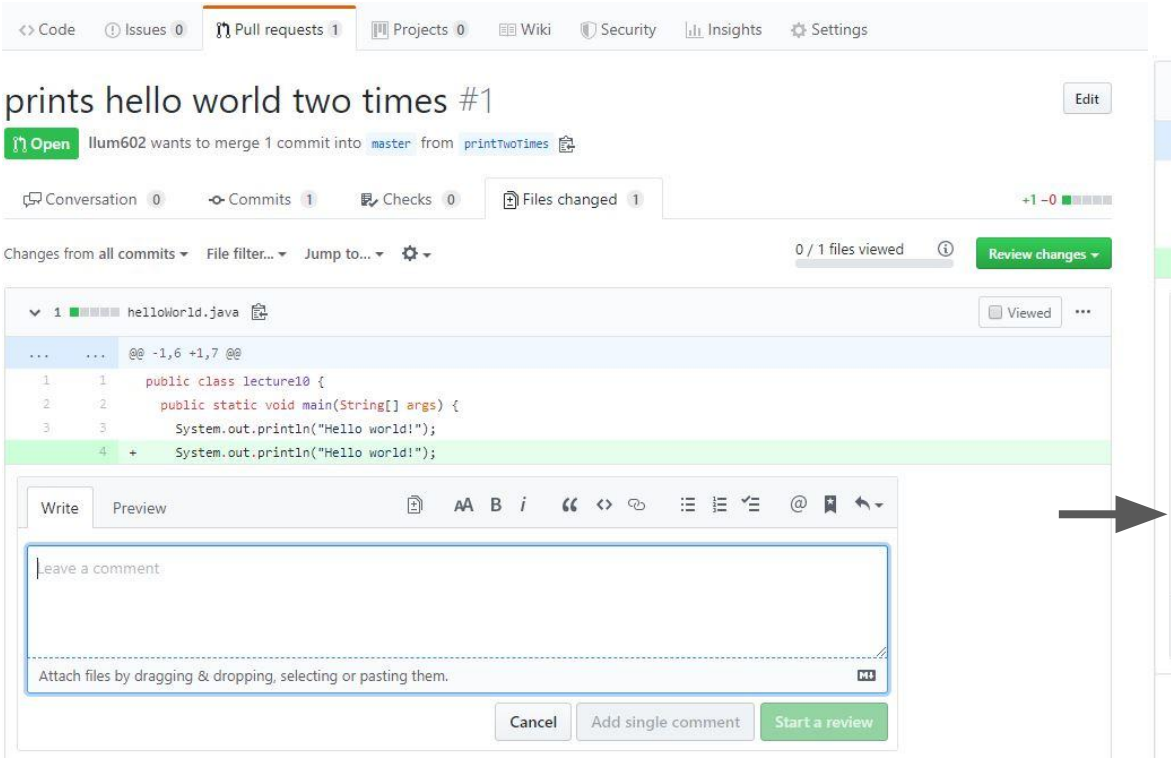
Branch you are merging **into**

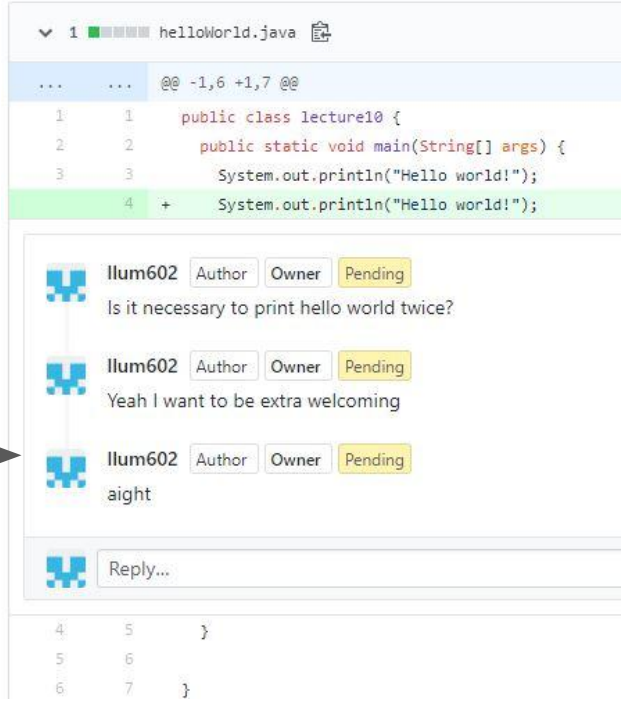
Feature branch

This means you have no merge conflicts

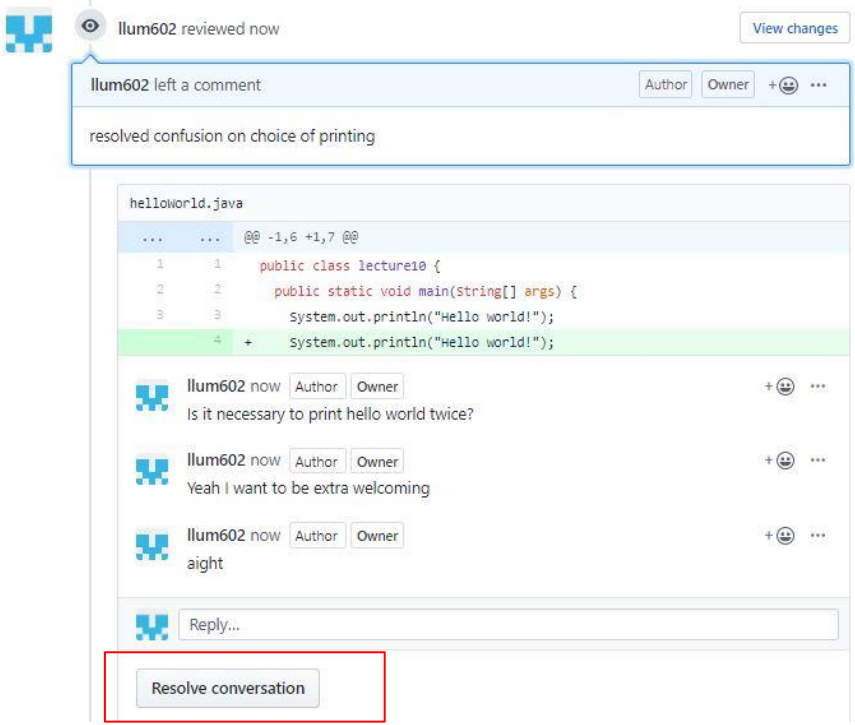


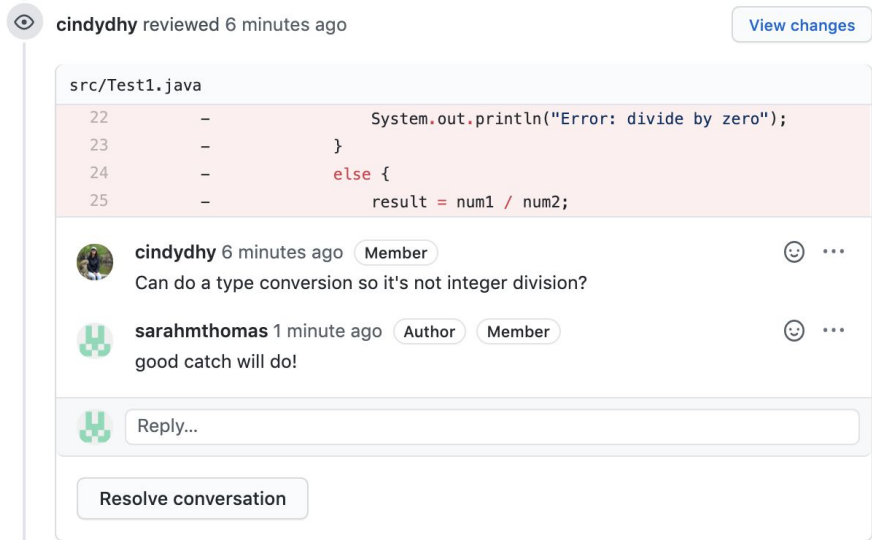
Commenting on a Pull Request





Resolving a Pull Request Conversation





Merging your Pull Request

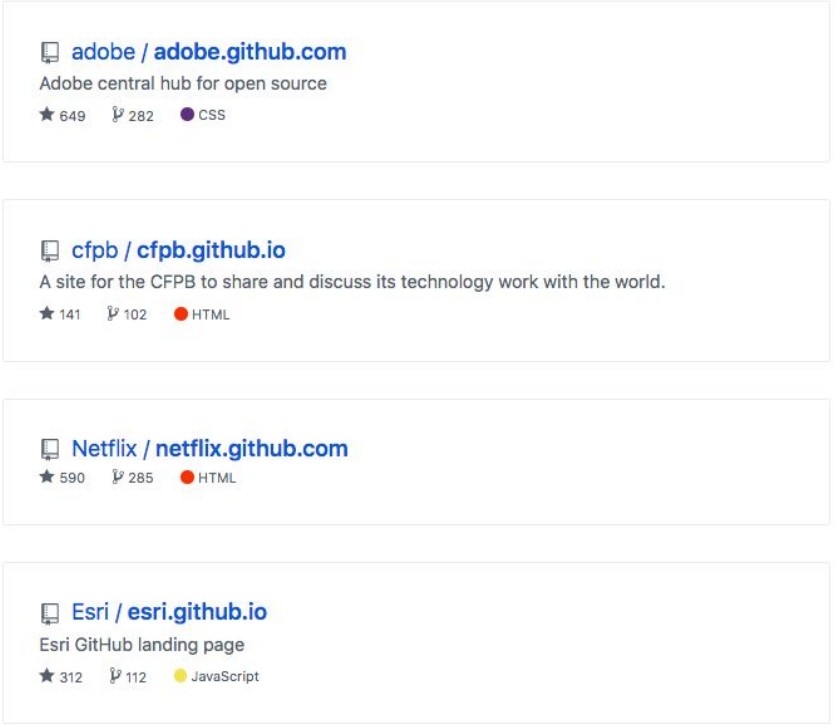


Note the “no conflicts” message!

GitHub Organizations

GitHub Organizations

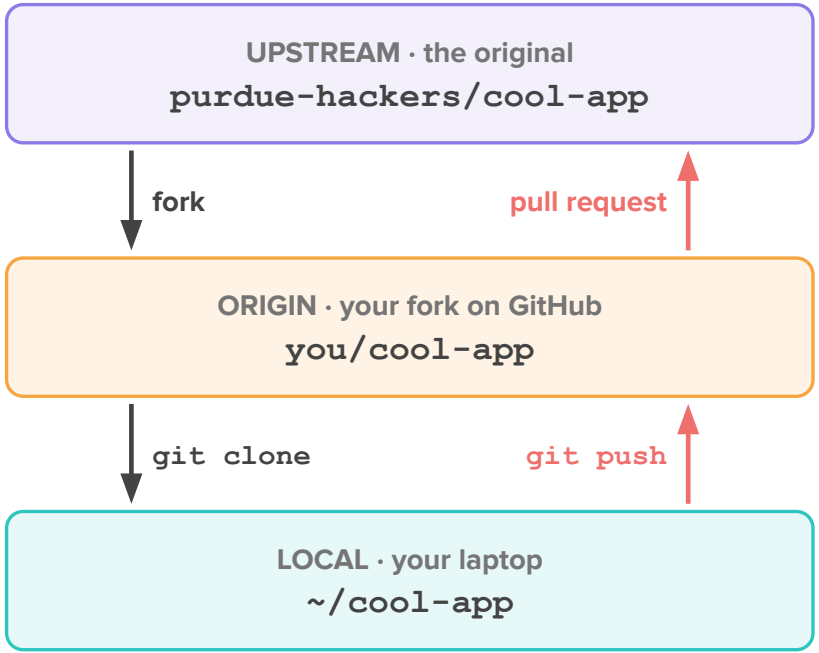
- Open source organizations
  - There are open source projects for any of your interests that you can contribute to
  - Purdue organizations
    - Purdue Hackers
    - USB
    - BoilerMake
    - And many more!!!



Forking

What is a fork?

- A fork is **your own copy** of someone else's repository, saved to your GitHub account
- It copies everything: files, branches, and the full commit history
- You can push to your fork freely. The original repo (the **upstream**) is untouched
- Want your changes in the original? Open a **pull request** from your fork
- This is how you contribute to open source projects you can't push to



Forking vs. Branching

	Branch	Fork
What it is	A separate line of commits inside one repo	A full copy of the repo under your account
Where it lives	In the same repository	In a new repository you own
Git or GitHub?	Git feature, works anywhere	GitHub (hosting) feature
Access needed	Write access to the repo	Just read access (any public repo)
Typical user	Teammates on the same project	Outside contributors, open source
Merging back	PR: feature branch → main	PR: your fork → upstream repo

You still branch inside a fork! Fork = your copy of the repo. Branch = one line of work inside a repo.

The fork workflow

1. **Fork the repo**
   Click “Fork” on the project’s GitHub page

   GitHub: Fork → Create fork

2. **Clone your fork**
   Use your copy’s URL, not the original

   ```
   git clone https://github.com/<you>/cool-app.git
   ```

3. **Branch, edit, commit**
   Same add + commit as always

   ```
   git checkout -b fix-typo
   ```

4. **Push to your fork**
   origin = your fork

   ```
   git push -u origin fix-typo
   ```

5. **Open a pull request**
   base: upstream/main ← head: you/fix-typo

   GitHub: Compare & pull request

Keeping your fork in sync

`origin`
Your fork. You push here.

`upstream`
The original. You fetch from here.

Your fork doesn't update itself when the original changes.

Shortcut: the "Sync fork" button on GitHub does the fetch + merge for you.

```
# once: connect to the original repo
git remote add upstream <original-url>

# whenever upstream has new commits
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

Review

Review

- Git is a Version Control System (VCS)
  - While not the only one, it is the most popular
  - You'll use it in classes and your career
  - History of changes
- Local vs Remote Repos
- It keeps tracks of changes you make to files using **COMMITS**
  - Easier to collaborate
  - Harder to lose work
- Each commit is reversible
- **ADD**, **COMMIT**, **PUSH**
- Git is p. cool + essential to know



git cheat sheet part 2

`git clone [url]`	Copy a remote repository to your local machine.
`git help [command]`	Display documentation for a Git command.
`git branch`	List all local branches. * marks the current branch.
`git branch [name]`	Create a new branch.
`git branch -d [name]`	Delete a branch.
`git checkout [branch]`	Switch to an existing branch.
`git checkout -b [branch]`	Create a new branch and switch to it immediately.
`git merge [branch]`	Merge the specified branch into your current branch.
`git rebase [branch]`	Move current branch commits on top of another branch.

git cheat sheet 3

--- MERGE A BRANCH INTO MAIN ---

```
git checkout main             Switch to main.
git pull                      Make sure main is up to date.
git merge [your-branch]       Merge your branch into main.
git push                      Push the updated main to remote.
git branch -d [your-branch]           Delete branch locally.
git push origin --delete [your-branch] Delete branch on remote.
```

Any Questions?

What would happen if you cloned a forked branch, forked it two more times, and cloned it again?

Why didn't Andy notice his toys moving places?

Why does the ball have eyes? (see slide 30)

What's the one rule of GitHub company dinners? (Everyone gets their own fork)

Demo

Quiz Time!

Resources

Resources for Learning Git

1 Official git website https://git-scm.com/learn

2 Git mastery - learning different levels of git in an interactive way
https://www.gitmastery.me/

3 Visualizing https://git-school.github.io/visualizing-git/